Lab No: 13

❈ Lab Title: Structures

- **Learning objectives:**

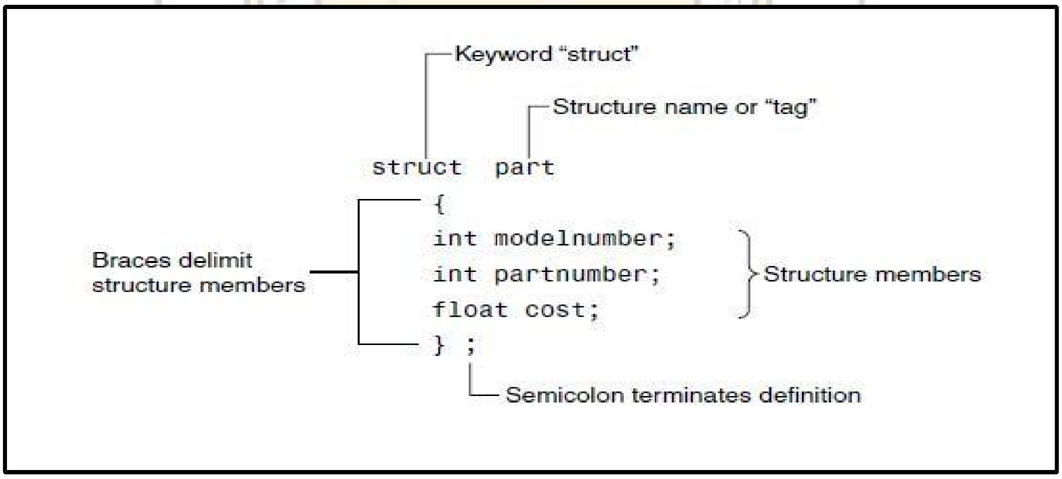
The following topics will be covered in this lab session

- What is a Structure
- When to use Structure
- Declaration of Structure
- Initialization of Structure
- Accessing Structure elements

STRUCTURES:

A Structure is a collection of simple variables. The variables in a structure can be of difference types: some can be int, some can be float, and so on (This is unlike the array, which we, ll meet later, in which all the variables must be the same type.) The data items in a structure are called the members of the structure.

Syntax:



- **Following points must be noted while declaring a structure type**:

The closing brace in the structure type declaration must be followed by a semicolon. It is important to understand that a structure type declaration does not tell the compiler to reserve any space in memory. All a structure declaration does is, it defines the 'form' of the structure. One structure can be nested within another structure. Like an ordinary variable, a structure variable can also be passed to a function. We may either pass individual structure elements or the entire structure variable at one go.

- **Example # 1:**

```
#include <iostream>
Using namespace std;
struct part                     //declare a structure
{
        int modelnumber;        //ID number of car
        int  partnumber;        //ID number of car part
            float cost;         //cost of part
};
int main()
{
                part part1;                 //define a structure variable
                part1.modelnumber = 6244; //give values to structure
members
                part1.partnumber = 373;
                part1.cost = 217.55F;       //display structure members
                cout << “Model “ << part1.modelnumber;
                cout << “, part “ << part1.partnumber;
                cout << “, costs $” << part1.cost << endl;
                return 0;

}
```

➪ Output:

```
Model 6244, part 373, costs $217.55
```

Lab Tasks

- **Task 1: Structure Declaration and Initialization:**

    1. Declare a structure called "Person" with the following members: "name" (string), "age" (integer), and "address" (string).
    2. Declare a variable called "person1" of type "Person" and initialize its members with values of your choice.
    3. Display the values of the members of "person1" using dot notation.

➪ Input:

```
#include <iostream>    // input/output ke liye library
using namespace std;    // standard namespace use kar rahe
hain

// Person structure define ho raha hai
struct Person {

    string name;     // name store karega
    int age;         // age store karega
    string address;  // address store karega
};

int main() {

    // person1 object banaya aur values initialize ki
    Person person1 = {"Ali", 20, "Frankfurt"};

    // output start
    cout << "Person Details:" << endl;

    // name print kar rahe hain
    cout << "Name: " << person1.name << endl;

    // age print kar rahe hain
    cout << "Age: " << person1.age << endl;

    // address print kar rahe hain
    cout << "Address: " << person1.address << endl;

    return 0; // program end
}
```

➯ **Output:**

Person Details:

Name: Ali Age:

20
Address: Frankfurt

- **Task 2: Structure Declaration and Initialization**:

  1. Declare a structure called "Student" with the following members:
     - Name (string)
     - Age (integer)
     - Grade (char)
  2. Declare a variable of type "Student" called "student1" and initialize its members with values of your choice.
  3. Display the values of the "student1" structure members.

Input:

```
#include <iostream>   // input/output library
using namespace std;  // standard namespace

// Student structure define ho raha hai
struct Student {

    string name;  // student ka naam
    int age;       // student ki age
    char grade;    // grade store karega (A, B, C)
};

int main() {

    // student1 object banaya aur initialize kiya
    Student student1 = {"Sara", 19, 'A'};

    // output start
    cout << "Student Details:" << endl;

    // name print
    cout << "Name: " << student1.name << endl;
```

```
   // age print
   cout << "Age: " << student1.age << endl;

   // grade print
   cout << "Grade: " << student1.grade << endl;

   return 0; // program end
}
```

➥ **Output:**

Student Details:
Name: Sara Age:
19
Grade: A

- **Task 3: Accessing Structure Elements:**

1. Declare a structure called "Book" with the following members:
    - Title (string)
    - Author (string)
    - Price (float)
    - Pages(integers)
2. Declare a variable of type "Book" called "book1"and books, and initialize its members with values of your choice.
3. Display values of structure members of varaiables **book1** and **book2**

➥ **Input:**

```
#include <iostream>   // input/output library
using namespace std;  // standard namespace

// Book structure define ho raha hai
struct Book {

    string title;   // book ka title
    string author;  // author ka naam
    float price;    // price store karega
    int pages;      // pages store karega
};

int main() {

    // book1 object initialize
    Book book1 = {"C++ Basics", "John Smith", 500.50, 300};

    // book2 object initialize
    Book book2 = {"Data Structures", "Mark Lee", 750.00, 450};

    // book1 output
    cout << "Book 1 Details:" << endl;

    cout << "Title: " << book1.title << endl;
    cout << "Author: " << book1.author << endl;
    cout << "Price: " << book1.price << endl;
    cout << "Pages: " << book1.pages << endl;
```

```
cout << "------------------------" << endl;

// book2 output
cout << "Book 2 Details:" << endl;

cout << "Title: " << book2.title << endl;
cout << "Author: " << book2.author << endl;
cout << "Price: " << book2.price << endl;
cout << "Pages: " << book2.pages << endl;

return 0; // program end
}
```

Output:

Book 1 Details: Title:

C++ Basics Author:

John Smith Price:

500.5

Pages: 300

Book 2 Details:

Title: Data Structures

Author: Mark Lee

Price: 750

Pages: 450

Conclusion:

In this lab, we learned how to use **structures in C++** to store and manage related data under a single user-defined data type. A structure allows us to group different types of variables (like `string`, `int`, `float`, and `char`) into one unit, making programs more organized and easier to understand.

We also learned how to:

- Declare a structure using the `struct` keyword
- Create structure variables (objects)
- Initialize structure members with values
- Access structure members using the dot operator (.)